

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO - FECAP

Bacharelado em Ciência da Computação

Sistemas operacionais e Computação em Nuvem – Rodrigo da Rosa

Estudantes:

- André Gregório dos Santos – RA: 24026489
- Guilherme Reis Fogolin de Godoy – RA: 24026241
- Pedro Henrique Nascimento Lemos – RA: 23025380
- Yan Ramos Cezareto – RA: 24026005

Turma: 5NACOMP_S

**Entrega 02 -
Sistema de Monitoramento de instância linux em um AWS EC2.**

1 INTRODUÇÃO

Para a Entrega 02 da disciplina de Sistemas Operacionais e Computação em Nuvem, o nosso grupo subiu uma aplicação Python rodando dentro de containers Docker em uma instância Linux na AWS. O foco do trabalho foi: provisionar uma máquina virtual em nuvem, configurar o sistema operacional, selecionar 4 exercícios utilizando IA e ML, para rodar na instância Linux, empacotar a aplicação em containers, fazer deploy remoto e monitorar o consumo de recursos (CPU, memória, disco e rede) tanto pela ferramenta de monitoramento nativa da AWS quanto pelo dashboard da própria aplicação.

A aplicação em si (chamada AbraceAI Analytics) usa quatro modelos de IA simples para enriquecer os dados coletados, mas isso é só o conteúdo que ela processa. O importante para esta disciplina é demonstrar todo o ciclo de operação: como a infraestrutura foi provisionada, quais comandos foram rodados no terminal Linux, como o Docker foi configurado, como o acesso SSH foi feito e quais métricas do sistema foram coletadas e visualizadas.

Tudo foi feito com a conta AWS de um dos integrantes do grupo, dentro do Free Tier da região us-east-1, em uma instância t3.micro com Ubuntu Server 24.04 LTS. Ao final, o ambiente foi totalmente destruído com um script de delete automatizado, garantindo custo final igual a US\$ 0,00.

2 OBJETIVOS DO TRABALHO

O objetivo principal foi cumprir o requisito da Entrega 02: subir uma aplicação em uma instância Linux na nuvem usando Docker. Os objetivos específicos, todos relacionados à disciplina, foram:

- Provisionar uma instância EC2 Ubuntu Server na AWS via Infraestrutura como Código (CloudFormation).
- Configurar Security Group, KeyPair e regras de rede para acesso seguro à máquina.
- Instalar e configurar o Docker e o Docker Compose no Ubuntu remoto.
- Empacotar a aplicação em containers e subir o stack na máquina via SSH.
- Acompanhar o consumo de recursos da instância pelo console AWS (CloudWatch) e pelo dashboard da aplicação.
- Encerrar todos os recursos no final, garantindo custo zero.

A aplicação que rodou na instância usa quatro modelos de Inteligência Artificial e Aprendizado de Máquina sobre as métricas coletadas resumidos na tabela a seguir.

Tipo de exercício	Algoritmos utilizados	O que faz com as métricas
Regressão	Random Forest Regressor (vs. Linear e Polinomial-Ridge)	Prevê o uso de CPU no próximo intervalo (cpu_percent_t+1) com base no histórico recente.
Classificação	Decision Tree e Random Forest (vs. Regressão Logística e KNN)	Classifica o estado atual da instância em normal, atenção ou crítico, com base nos limites de CPU e memória.
Clusterização	K-Means (k=2) e DBSCAN	Agrupa os instantes coletados em perfis de uso (ex.: ocioso vs. carregado) sem precisar de rótulos prévios.
Redução de dimensionalidade	PCA (2 componentes principais)	Reduz as 8 métricas numéricas a 2 dimensões para visualizar os clusters em um gráfico 2D (95,6% de variância explicada).

Tabela 1 – Modelos de IA/ML aplicados sobre as métricas da instância EC2

3 VISÃO GERAL DA SOLUÇÃO

A solução é composta por três peças que rodam dentro da mesma instância EC2, todas dentro de containers Docker:

- Coletor: processo Python em loop infinito que lê CPU, memória, disco, rede e load average a cada 5 segundos e grava em um CSV diário.
- Trainer: processo de execução manual que lê o CSV, treina os modelos de IA e salva os resultados em arquivos joblib.
- Dashboard: interface web em Streamlit (porta 8501) que mostra os gráficos das métricas em tempo real e os resultados dos modelos.

A stack tecnológica usada foi:

- Sistema operacional: Ubuntu Server 24.04 LTS (AMI oficial da Canonical).
- Containerização: Docker Engine 29.x + Docker Compose v2.
- Linguagem da aplicação: Python 3.12 (imagem base python:3.12-slim).
- Provedor de nuvem: AWS (região us-east-1, Free Tier).
- Infraestrutura como Código: AWS CloudFormation (template YAML).

- Acesso remoto: SSH com KeyPair RSA gerada via AWS CLI.
- Bibliotecas Python da aplicação: psutil (métricas do SO), pandas, scikit-learn, Streamlit.

4 AMBIENTE DE DESENVOLVIMENTO LOCAL

Toda a parte de desenvolvimento foi feita nas máquinas dos integrantes do grupo, antes de subir qualquer coisa na nuvem. A vantagem de fazer primeiro local é que conseguimos validar que o Dockerfile, o docker-compose.yml e os volumes compartilhados funcionavam, sem ficar depurando diretamente na EC2 (o que sairia caro em tempo).

4.1 Build e execução com Docker Compose

O projeto foi montado com um Dockerfile multi-stage (imagem final em torno de 600 MB, com Python 3.12-slim como base) e um docker-compose.yml que define os três serviços (collector, dashboard e trainer) compartilhando os mesmos volumes de data e reports. O comando para subir o stack inteiro localmente é:

```
cp .env.example .env
docker compose up -d --build
```

Para verificar o estado dos containers, conferir os logs e parar tudo:

```
docker compose ps
docker compose logs -f collector
docker compose down
```

Como toda a configuração da aplicação vem de variáveis de ambiente definidas no .env (caminhos de pasta, intervalo de coleta, porta do dashboard, limiares de classificação), o mesmo container que roda local funciona igual na EC2 sem alterar nenhuma linha de código. Essa portabilidade é justamente o que o Docker entrega.

5 PROVISIONAMENTO DA INFRAESTRUTURA NA AWS

5.1 Pré-requisitos: AWS CLI, KeyPair e Budget Alert

Antes de qualquer deploy, configuramos o AWS CLI na máquina local com as credenciais da conta. A verificação básica é:

```
aws sts get-caller-identity
```

A saída mostra o ID da conta, o ARN do usuário e o UserId. Se esse comando responde, o restante do deploy via CloudFormation funciona.

Em seguida, criamos a KeyPair (par de chaves RSA) que será usada para abrir conexões SSH na instância. A AWS guarda apenas a chave pública; a privada (.pem) é baixada na hora da criação e tem que ser protegida com `chmod 400` (caso contrário o cliente SSH recusa usá-la):

```
aws ec2 create-key-pair --key-name abrace-ai-analytics \
  --query 'KeyMaterial' --output text \
  > ~/Downloads/abrace-ai-analytics.pem
chmod 400 ~/Downloads/abrace-ai-analytics.pem
```

Como salvaguarda contra esquecer recursos ligados, configuramos um Budget Alert de US\$ 1 na conta, com notificação por e-mail aos 80% do limite. Esse passo é importante porque o maior risco do trabalho não é técnico, é financeiro: deixar uma EC2 ligada por engano. Com o Budget Alert, mesmo que esquecêssemos a stack ligada por um mês inteiro (cerca de US\$ 7,50 fora do Free Tier), receberíamos o aviso antes do gasto se materializar.

5.2 Template CloudFormation

Toda a infraestrutura foi descrita em um único arquivo YAML (infra/cloudformation.yml), o que evita ter que clicar manualmente no console e garante reprodutibilidade. O template provisiona dois recursos principais:

- Um Security Group (firewall virtual) liberando apenas as portas 22 (SSH) e 8501 (dashboard) para o IP público do operador. Em nenhum momento usamos 0.0.0.0/0.
- Uma instância EC2 t3.micro com Ubuntu Server 24.04 LTS, EBS gp3 de 12 GB criptografado, atrelada à KeyPair criada e ao Security Group acima.

A AMI do Ubuntu é resolvida dinamicamente via SSM Parameter da Canonical, ou seja, em vez de fixar um ID de imagem (que muda quando sai uma versão nova), o template aponta para um caminho oficial da Canonical e a AWS substitui pela AMI mais recente automaticamente. O trecho do template fica assim:

```
UbuntuAmiSsmParameter:
  Type: AWS::SSM::Parameter::Value<AWS::EC2::Image::Id>
  Default: /aws/service/canonical/ubuntu/server/24.04/\
    stable/current/amd64/hvm/ebs-gp3/ami-id
```

Na primeira tentativa de deploy, deu um erro de YAML mal-formatado (linha 25, coluna 52). O parser do CloudFormation interpretava o trecho 'ex.: 1.2.3.4' presente na descrição de um parâmetro como separador de chave e valor (porque YAML usa ':' para isso). A correção foi colocar a descrição inteira entre aspas duplas, o que força o tratamento como string única. Depois disso o template validou e a stack subiu.

5.3 Execução do deploy

O script `infra/deploy_aws.sh` automatiza todo o deploy: ele confirma o login no AWS CLI, pega o IP público da máquina local automaticamente (via `curl` em `checkip.amazonaws.com`), pergunta o nome da KeyPair e dispara o `aws cloudformation deploy`. O comando resumido é:

```
bash infra/deploy_aws.sh
```

A criação da stack levou cerca de 5 minutos. Durante esse tempo, o CloudFormation cria o Security Group, lança a instância, anexa o EBS, associa o IP público e roda o script de bootstrap definido no UserData. Ao final, mostra os Outputs com o DNS público da máquina, o comando SSH pronto para colar e a URL do dashboard. A instância ficou visível no console como running:

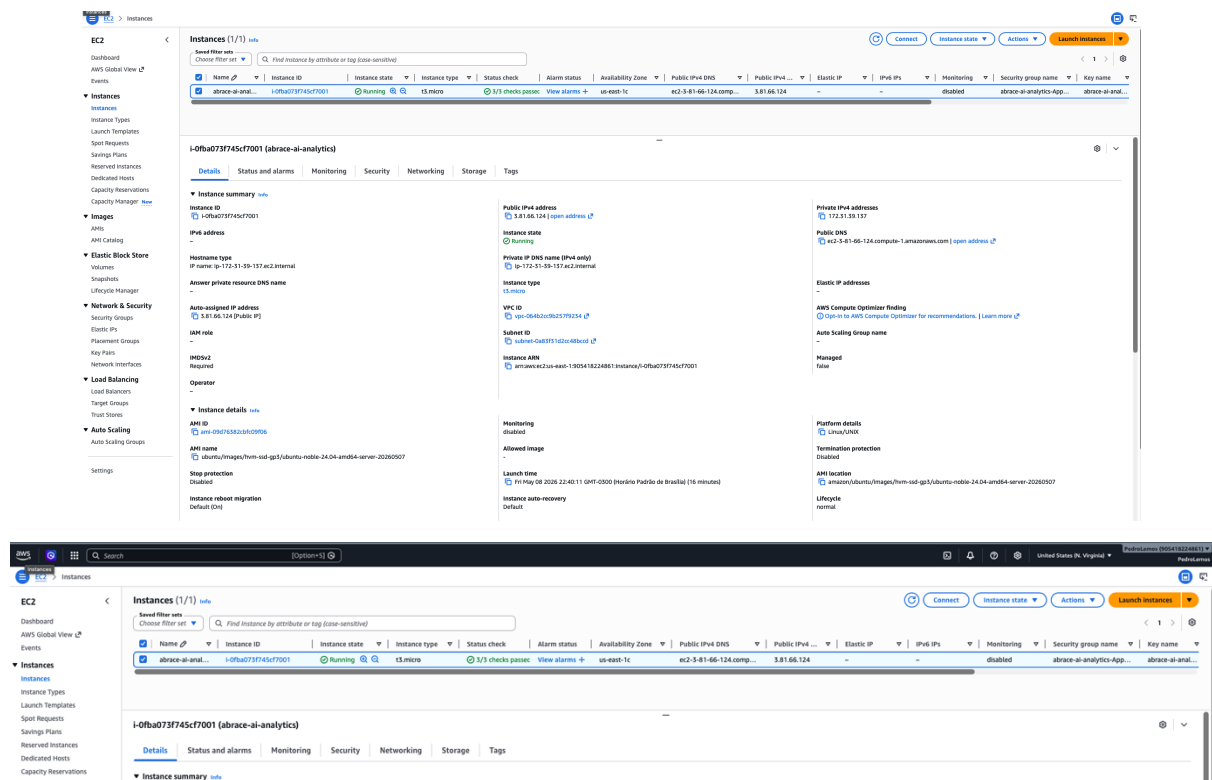


Figura 1 – Console da AWS mostrando a instância EC2 t3.micro provisionada e em estado running

6 CONFIGURAÇÃO DO LINUX REMOTO (UBUNTU 24.04)

6.1 Bootstrap automatizado via UserData

Quando uma instância EC2 é criada, ela aceita um script de inicialização chamado UserData que roda automaticamente como root na primeira boot. Aproveitamos esse

mecanismo para deixar o Ubuntu pronto para receber a aplicação sem precisar instalar nada manualmente depois pelo SSH. O script faz:

- `apt-get update + apt-get upgrade -y` para deixar o sistema atualizado.
- Instalação de `git`, `curl`, `ca-certificates` e `gnupg`.
- Adição do repositório oficial do Docker (chave GPG da Canonical).
- Instalação do Docker Engine, Docker CLI, `containerd`, `buildx` e Docker Compose plugin.
- `usermod -aG docker ubuntu` (para o usuário `ubuntu` poder rodar docker sem `sudo` nas próximas sessões).
- `systemctl enable --now docker` (para o Docker subir junto com a máquina e estar ativo agora).
- Criação da pasta `/opt/abrace-ai-analytics` com o dono `ubuntu`.

Toda a saída do bootstrap fica gravada em `/var/log/cloud-init-output.log`, o que é fundamental para auditoria: se alguma coisa falhar na instalação, é nesse arquivo que aparece o erro. Quando tudo dá certo, ele termina com a mensagem `[abrace-ai] bootstrap finalizado`.

6.2 Conexão SSH na instância

Com a stack pronta, conectamos na instância usando o comando SSH que veio nos Outputs do CloudFormation. O usuário do Ubuntu Server na AWS é `ubuntu` (não `root`), e a autenticação é por chave (a senha fica desabilitada por padrão):

```
ssh -i ~/Downloads/abrace-ai-analytics.pem \
ubuntu@ec2-3-81-66-124.compute-1.amazonaws.com
```

Na primeira conexão, o cliente SSH pediu confirmação do fingerprint `ED25519` da host key da máquina (mecanismo de proteção contra `man-in-the-middle`). Respondemos `yes` e o fingerprint foi gravado em `~/.ssh/known_hosts`; a partir daí, conexões seguintes não pedem mais.

6.3 Verificação do bootstrap

Já dentro da máquina, conferimos se o bootstrap tinha terminado bem. O log do `cloud-init` guarda toda a saída do `UserData`:

```
sudo tail -50 /var/log/cloud-init-output.log
```

Vimos no final do log a mensagem `[abrace-ai] bootstrap finalizado`, confirmando que Docker, Docker Compose, `git` e `curl` foram instalados sem erro e que a pasta `/opt/abrace-ai-`

analytics estava pronta para receber o projeto. Também verificamos se o serviço do Docker estava ativo:

```
systemctl status docker
docker --version
docker compose version
```

7 DEPLOY DA APLICAÇÃO NA INSTÂNCIA

7.1 Envio do código via rsync

Como deixamos o `GitRepoUrl` vazio na hora do deploy (não quisemos fazer git clone dentro da máquina), o `/opt/abrace-ai-analytics` na EC2 ficou vazio. Para enviar o código da nossa máquina local para a instância, usamos rsync sobre SSH. O rsync é mais inteligente que o scp porque consegue ignorar pastas pesadas (cache do Python, dados brutos, modelos antigos) e só transfere o que mudou em execuções futuras:

```
DNS=$(aws cloudformation describe-stacks \
  --stack-name abrace-ai-analytics --region us-east-1 \
  --query
"Stacks[0].Outputs[?OutputKey=='PublicDnsName'].OutputValue" --
output text)

rsync -avz --delete \
  -e "ssh -i ~/Downloads/abrace-ai-analytics.pem" \
  --exclude '.git' --exclude '.venv' --exclude '__pycache__' \
  --exclude 'data/raw/*' --exclude 'data/processed/*' \
  --exclude 'data/models/*' --exclude 'reports/figures/*' \
  ./ "ubuntu@${DNS}:/opt/abrace-ai-analytics/"
```

Em poucos segundos o código foi todo replicado para a EC2, mantendo as mesmas pastas, mas sem o que é gerado em runtime. Esse pode ser considerado o equivalente a um “deploy” simples, sem pipeline de CI/CD, mas que demonstra o caminho de envio de artefatos para a nuvem.

7.2 Subindo os containers no Ubuntu

De volta na sessão SSH, criamos o arquivo `.env` (cópia do `.env.example`) e subimos o stack com Docker Compose. Como o usuário `ubuntu` foi adicionado ao grupo `docker` durante o bootstrap, mas a alteração só passa a valer numa nova sessão SSH, a primeira vez rodamos com `sudo`:

```
cd /opt/abrace-ai-analytics
sudo docker compose up -d --build
```


O primeiro build levou alguns minutos: o Docker baixou a imagem base do Python, instalou as dependências do requirements.txt e construiu a imagem da aplicação. Ao final, os containers subiram em background e ficaram em estado healthy.

```
ubuntu@ip-172-31-39-137:~$ cd /opt/abrace-ai-analytics/
ubuntu@ip-172-31-39-137:/opt/abrace-ai-analytics$ [-f .env ] || cp .env.example .env
sudo docker compose up -d --build
[+] up 1/1
x Image abrace-ai-analytics:latest Error pull access denied for abrace-ai-analytics, repository does not exist or may require 'docker login' 0.2s
[+] Building 118.5s (24/24) FINISHED
=> [internal] load local bake definitions 0.0s
=> => reading from stdin 540B 0.0s
=> [internal] load build definition from Dockerfile 0.0s
=> => transferring Dockerfile: 2.21kB 0.0s
=> resolve image config for docker-image://docker.io/docker/dockerfile:1.7 0.4s
=> docker-image://docker.io/docker/dockerfile:1.7sha256:a57df69d0ea827fb7266491f2813635de6f17269be881f696fbfd2d83dda33e 0.4s
=> resolve docker.io/docker/dockerfile:1.7sha256:a57df69d0ea827fb7266491f2813635de6f17269be881f696fbfd2d83dda33e 0.0s
=> sha256:96018c57e425989b7f10c074d80672ecdbd3bb7dcd38c1bd95960cf291207416 11.98MB / 11.98MB 0.2s
=> => extracting sha256:96018c57e425989b7f10c074d80672ecdbd3bb7dcd38c1bd95960cf291207416 0.2s
=> [internal] load metadata for docker.io/library/python:3.12-slim 0.4s
=> [internal] load .dockerignore 0.0s
=> => transferring context: 458B 0.0s
=> [internal] load build context 0.1s
=> => transferring context: 74.85kB 0.0s
=> [builder 1/5] FROM docker.io/library/python:3.12-slimsha256:6026d9374020066a85690cabdb66f5d06a2dd606e756c7082fcdcaaf6d048dd 2.2s
=> => resolve docker.io/library/python:3.12-slimsha256:6026d9374020066a85690cabdb66f5d06a2dd606e756c7082fcdcaaf6d048dd 0.0s
=> sha256:b33ff618953dc6b177e78bc33883a49817e571a1d9f0a3aa039e3228e0f21684 250B / 250B 0.0s
=> sha256:79789503061a7d1332e7b5c3df030896846f0a6a179f90060e85564dca1cf65 1.29MB / 1.29MB 0.1s
=> sha256:750aeb39a86e02ccc351143d34417e4b196c9684240898a0e3b8ec13a9 12.11MB / 12.11MB 0.2s
=> sha256:57fb7124685257a374deb7564ceca10f43c235272b501efc08add5d24ebb61 29.78MB / 29.78MB 0.4s
=> => extracting sha256:57fb7124685257a374deb7564ceca10f43c235272b501efc08add5d24ebb61 1.0s
=> => extracting sha256:79789503061a7d1332e7b5c3df030896846f0a6a179f90060e85564dca1cf65 0.1s
=> => extracting sha256:750aeb39a86e02ccc351143d34417e4b196c9684240898a0e3b8ec13a9 0.0s
=> => extracting sha256:b33ff618953dc6b177e78bc33883a49817e571a1d9f0a3aa039e3228e0f21684 0.0s
=> [builder 2/5] RUN apt-get update && apt-get install -y --no-install-recommends build-essential gcc && rm -rf /var/lib/apt/lists/* 17.1s
=> [runtime 2/11] WORKDIR /app 0.1s
=> [runtime 3/11] RUN apt-get update && apt-get install -y --no-install-recommends curl && rm -rf /var/lib/apt/lists/* 0.1s
=> [builder 3/5] WORKDIR /build 0.1s
=> [builder 4/5] COPY requirements.txt . 0.1s
=> [builder 5/5] RUN pip install --upgrade pip && pip wheel --wheel-dir /wheels -r requirements.txt 13.0s
=> [runtime 4/11] COPY --from=builder /wheels /wheels 1.0s
=> [runtime 5/11] COPY requirements.txt 0.1s
=> [runtime 6/11] RUN pip install --no-index --find-links=/wheels -r requirements.txt && rm -rf /wheels 28.2s
=> [runtime 7/11] RUN useradd --create-home --shell /usr/sbin/nologin abrace 0.5s
=> [runtime 8/11] COPY app /app 0.1s
=> [runtime 9/11] COPY scripts /scripts 0.1s
=> [runtime 10/11] COPY docker/entrypoint.sh /usr/local/bin/entrypoint.sh 0.1s
=> [runtime 11/11] RUN chmod +x /usr/local/bin/entrypoint.sh && mkdir -p /app/data/raw /app/data/processed /app/data/models /app/reports/figures /app/reports/m 0.3s
=> => exporting to image 52.3s
=> => exporting layers 42.6s
=> => exporting manifest sha256:d8892d54d8547b1a354e4e85133b9e27b9f900c485c3af9fe1508580d630808a 0.0s
=> => exporting config sha256:0d60419b3b2d226f3dd8345ccee8b47b29f22e6ad73999487da7b759247d82c 0.0s
=> => exporting attestation manifest sha256:34e03cb4e727067c8a60a103e7cab8923e41c0c08a701e42bf4e25a346d763bb 0.0s
=> => exporting manifest list sha256:cf019c0b7ea22084f46e134958c26e30124c262a0b3d27586a0e3b7fcd5 0.0s
=> => naming to docker.io/library/abrace-ai-analytics:latest 0.0s
[+] up 4/4acking to docker.io/library/abrace-ai-analytics:latest 9.5s
x Image abrace-ai-analytics:latest Built 119.3s
x Network abrace-ai-analytics_default Created 0.1s
x Container abrace_collector Started 0.0s
x Container abrace_dashboard Started 0.9s
```

Figura 2 – Sessão SSH na instância EC2 mostrando o docker compose up -d --build em execução

7.3 Verificação dos containers em execução

Para confirmar que tudo subiu certinho, usamos os comandos básicos de gerenciamento do Docker Compose:

```
sudo docker compose ps
sudo docker compose logs --tail 30 collector
sudo docker compose logs --tail 30 dashboard
```

O collector mostrou no log que estava gravando uma linha nova de métricas a cada 5 segundos no CSV diário, e o dashboard mostrou que o servidor Streamlit estava ativo na porta 8501.

8 MONITORAMENTO DA INSTÂNCIA EM PRODUÇÃO

Esta seção é o coração do trabalho do ponto de vista da disciplina: uma vez que a aplicação está rodando na nuvem, como acompanhamos o consumo de recursos do sistema operacional? Usamos duas fontes de monitoramento em paralelo: a aba Monitoring nativa da AWS (que vem do CloudWatch) e o dashboard da própria aplicação (que lê os mesmos tipos de métrica usando psutil).

8.1 Métricas no console AWS (CloudWatch)

O console da AWS oferece, para cada instância EC2, uma aba Monitoring com gráficos de várias métricas coletadas automaticamente pelo CloudWatch: CPUUtilization (uso de CPU em %), NetworkIn / NetworkOut (tráfego de rede em bytes), DiskReadOps / DiskWriteOps (operações de I/O no disco), entre outras. Essas métricas são coletadas em intervalos de 5 minutos no Free Tier e ficam disponíveis para gráficos e alarmes sem precisar instalar nada na máquina.

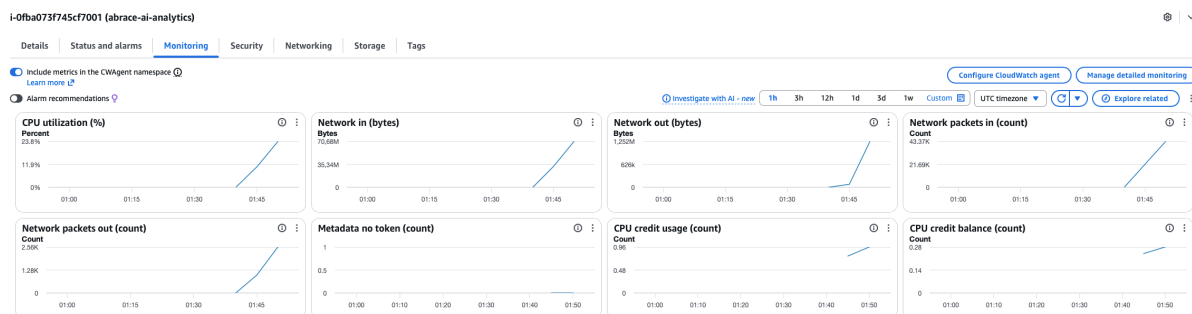


Figura 3 – Aba Monitoring da instância EC2 no console AWS, com métricas de CPU, rede e disco coletadas pelo CloudWatch

Dá para ver nos gráficos o aumento de uso de CPU correspondente ao período em que o coletor da aplicação estava rodando, validando que o sistema operacional realmente estava trabalhando dentro do container Docker.

8.2 Dashboard da aplicação em tempo real

Em paralelo ao CloudWatch, o dashboard da aplicação (Streamlit na porta 8501) mostra as mesmas métricas, mas com granularidade muito maior (uma amostra a cada 5 segundos, em vez de 5 minutos do CloudWatch) e atualização automática a cada 10 segundos. Ele lê os dados que o coletor está gravando no CSV em tempo real e plota gráficos de série temporal de CPU, memória, disco e memória disponível, todos na mesma tela.

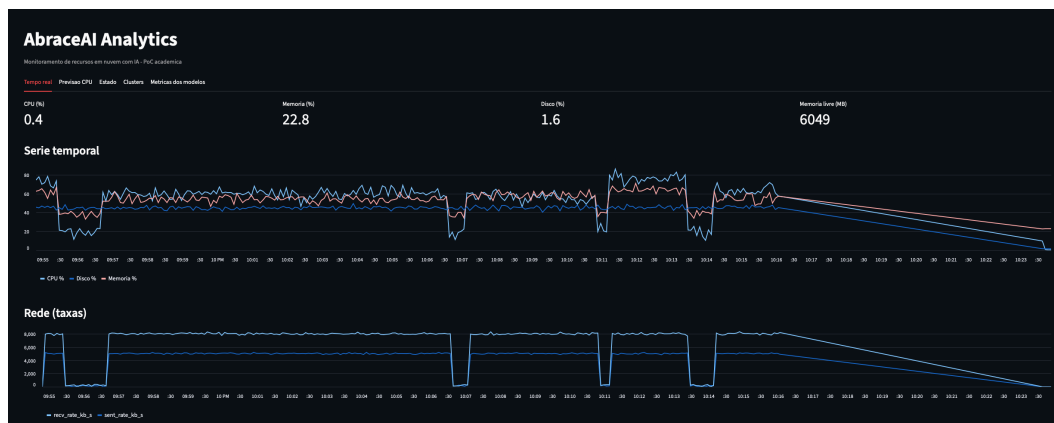


Figura 4 – Aba Tempo Real do dashboard mostrando séries temporais de CPU, memória, disco e memória livre lidas via psutil dentro do container

A diferença entre as duas fontes vale comentar: o CloudWatch monitora a instância EC2 inteira do lado de fora (do hipervisor da AWS), enquanto o psutil que roda dentro do container monitora o sistema operacional do ponto de vista do processo. Os números batem em geral, mas o dashboard interno tem mais detalhe (por exemplo, memória livre em MB, que o CloudWatch não mostra de graça).

O acesso ao dashboard ficou em <http://<DNS-público>:8501>. Como o Security Group restringe a porta 8501 ao mesmo IP autorizado para SSH, apenas as máquinas do nosso grupo conseguiram acessar a interface — qualquer outra origem recebe timeout, o que demonstra a regra de firewall funcionando como esperado.

8.3 Aplicação dos modelos e análise de desempenho

Os dados coletados pelo psutil dentro da instância serviram de entrada para quatro exercícios de IA/AM com scikit-learn, atendendo o requisito do enunciado (mínimo de três exercícios da disciplina de Inteligência Artificial e Aprendizado de Máquina). O dataset usado no treino foi de 8.640 amostras (12 horas a uma amostra a cada 5 segundos), com split temporal 80/20 (sem shuffle, para preservar a natureza de série temporal). Todos os modelos foram treinados com `random_state=42` para reprodutibilidade. As subseções a seguir descrevem cada exercício e ao final está a tabela com a comparação de desempenho.

8.3.1 Regressão para previsão de CPU

Variável alvo: `cpu_percent_t+1` (uso de CPU no próximo intervalo de 5 segundos).
Features: medições atuais de CPU, memória, disco, rede, load average e médias móveis de CPU em janelas de 1 e 5 minutos. Comparamos três modelos: Regressão Linear, Regressão

Polinomial de grau 2 com Ridge (regularizada) e Random Forest Regressor. As métricas avaliadas foram MAE, MSE, RMSE e R^2 .

O Random Forest Regressor venceu com $R^2 = 0,750$ e MAE = 8,24 pontos percentuais. A regressão polinomial regularizada teve R^2 negativo (sinal de overfitting), e a linear ficou em $R^2 = 0,613$. O melhor modelo foi salvo em data/models/regression_best.joblib.

Figura 5 – Previsão de CPU (Random Forest Regressor) sobreposta à série real, exibida na aba Previsão CPU do dashboard

8.3.2 Classificação do estado da instância

Variável alvo: risk_label, com três classes (normal, atenção, crítico) atribuídas pelos limiares definidos no .env (CPU < 50% e memória < 60% = normal; CPU > 80% ou memória > 80% = crítico; intermediário = atenção). Comparamos quatro classificadores: Regressão Logística, KNN, Árvore de Decisão e Random Forest, com validação cruzada de 5 folds e avaliação por acurácia, precisão (macro), recall (macro), F1 (macro e ponderado).

Decision Tree e Random Forest empataram com F1-macro de 0,998, porque a rotulagem é baseada em regras explícitas de limiar e os modelos baseados em árvore aprendem esse tipo de regra com facilidade. A Regressão Logística obteve F1-macro de 0,695 e o KNN ficou em 0,580 (mais sensível à escala das features e ao ruído).

8.3.3 Clusterização com K-Means e DBSCAN

Aprendizado não supervisionado: o algoritmo agrupa as amostras em perfis sem usar rótulo. Aplicamos K-Means com análise de cotovelo e coeficiente de silhueta para escolher o melhor k (resultado: k=2, silhouette = 0,646). Os dois clusters separam claramente um perfil ocioso (CPU média de 27,6%, load 1,11) de um perfil carregado (CPU média de 73,8%, load 2,95). Em paralelo, rodamos DBSCAN com eps = 0,241, que encontrou 8 subgrupos e identificou 943 pontos como ruído.

Figura 6 – Análise de cotovelo e coeficiente de silhueta para escolha do número de clusters do K-Means

8.3.4 PCA para redução de dimensionalidade

PCA (Principal Component Analysis) reduz as 8 features numéricas a 2 dimensões mantendo o máximo de variância dos dados. Os dois primeiros componentes principais explicaram 95,6% da variância total (PC1 com 77,4% e PC2 com 18,1%), o que indica que o problema tem dimensionalidade intrínseca baixa, dominada pela carga geral da máquina. O

resultado foi usado para visualizar os clusters do K-Means e do DBSCAN num gráfico 2D legível.

Figura 7 – Visualização 2D dos clusters do K-Means e do DBSCAN no espaço PCA (PC1 + PC2 = 95,6% da variância)

8.3.5 Síntese de desempenho dos modelos

A tabela a seguir resume as métricas dos melhores modelos de cada exercício, calculadas sobre o conjunto de teste (1.728 amostras).

Exercício	Melhor modelo	Métricas obtidas
Regressão (CPU futura)	Random Forest Regressor	$R^2 = 0,750$ / MAE = 8,24 / RMSE = 12,47
Classificação (estado)	Decision Tree	Acurácia = 0,998 / F1-macro = 0,998 / CV-F1 = 1,000
Clusterização	K-Means (k = 2)	Silhouette = 0,646 (clusters: ocioso vs. carregado)
Redução de dimensionalidade	PCA (2 componentes)	Variância explicada = 95,6% (PC1 = 77,4% + PC2 = 18,1%)

Tabela 2 – Síntese de desempenho dos modelos de IA/ML aplicados

Ressalva importante: o dataset usado no treino é sintético (gerado por `scripts/seed_synthetic.py`) com perfis de carga controlados, e as classes da classificação são definidas por regras de limiar. Isso explica porque os modelos baseados em árvore atingem praticamente 100% de acerto na classificação — eles estão aprendendo exatamente a função geradora dos rótulos. Em um cenário com dados reais e ruído operacional, esses números seriam menores. Mesmo assim, a comparação relativa entre modelos continua válida e mostra o ganho de algoritmos não lineares (Random Forest) sobre lineares (Regressão Linear / Logística) para esse tipo de problema.

9 ENCERRAMENTO E CONTROLE DE CUSTOS

Depois de tirar todos os prints de evidência, o ambiente foi totalmente destruído com o script de teardown:

```
bash infra/teardown_aws.sh
```

O script pede confirmação manual (precisa digitar `yes`), executa o comando `aws cloudformation delete-stack`, aguarda o `stack-delete-complete` e ainda lista volumes EBS, Elastic IPs e snapshots remanescentes para conferência. No nosso caso, o CloudFormation removeu tudo de uma vez (instância, Security Group, EBS) e nenhuma sobra apareceu na lista.

O custo total da operação ficou em US\$ 0,00, conforme planejado, porque a conta estava no Free Tier e o tempo de execução da stack foi de poucas horas.

Esse passo é importante de ser documentado porque resume uma das boas práticas mais valiosas em computação em nuvem: provisionar apenas o necessário, monitorar com alerta de orçamento e destruir tudo após o uso. É o princípio básico de FinOps aplicado ao nosso trabalho de faculdade.

Resumo dos principais comandos usados ao longo do trabalho:

Etapas	Comando principal
Verificar AWS CLI	<code>aws sts get-caller-identity</code>
Criar KeyPair	<code>aws ec2 create-key-pair --key-name abrace-ai-analytics ...</code>
Provisionar stack	<code>bash infra/deploy_aws.sh</code>
Conectar na EC2	<code>ssh -i ~/Downloads/abrace-ai-analytics.pem ubuntu@<DNS></code>
Conferir bootstrap	<code>sudo tail -50 /var/log/cloud-init-output.log</code>
Enviar projeto	<code>rsync -avz -e "ssh -i ..." ./ ubuntu@<DNS>:/opt/abrace-ai-analytics/</code>
Subir containers	<code>sudo docker compose up -d --build</code>
Destruir stack	<code>bash infra/teardown_aws.sh</code>

Tabela 3 – Resumo dos comandos executados ao longo do trabalho

10 CONCLUSÃO

O trabalho cumpriu o que a disciplina de Sistemas Operacionais e Computação em Nuvem pediu: provisionamos uma máquina virtual Linux na AWS, configuramos o sistema operacional para receber a aplicação, subimos containers Docker via SSH, rodamos 4 modelos de I.A. aprendido na matéria de Inteligência Artificial e Aprendizado de Máquina, monitoramos o consumo de recursos do sistema (CPU, memória, disco e rede) tanto pelo CloudWatch quanto por um dashboard interno e, no final, destruimos tudo de forma automatizada para garantir custo zero.

Os pontos que mais ficaram claros para a gente como aprendizado de operação em nuvem foram: o cuidado com a segurança de rede em Computação na Nuvem, o uso correto do

SSH, sudo, o entendimento do ciclo de vida do Docker no Ubuntu (instalação, grupo, systemctl, compose) e o como monitorar métricas valiosas de instancias como: CPU, Memoria, Latência e entre outros, é crucial para entender como o nosso sistema desempenha em diversas situações.

